UNIVERSIDADE DE COIMBRA- FACULDADE DE CIÊNCIAS E TECNOLOGIA

Departamento de Engenharia Informática

Unidade curricular: Sistemas Distribuídos

Trabalho realizado por:

Beatriz Filipe Teixeira Fernandes (2023215703)

Diana Isabel De Sá Janeiro e Martins (2023218119)

Introdução

O projeto BeeCurious_SD consiste numa aplicação que integra diferentes tecnologias (Java RMI, Spring Boot, APIs REST) para realizar pesquisas na internet. Funciona de forma semelhante a um motor de busca tradicional, permitindo indexar URLs, pesquisar palavras, etc. O sistema está dividido em duas grandes partes:

· Backend

· Frontend

Arquitetura do Software

O sistema segue uma arquitetura cliente-servidor distribuída com os seguintes componentes principais:

- Frontend: Aplicação web Spring Boot com Thymeleaf
- Backend: Sistema distribuído com RMI (Gateway, Barrels, Downloaders e Queue)
- Integrações externas: Hacker News API e OpenRouter AI

Gateway

Descrição geral:

A gateway é o componente central do sistema, tendo o papel de intermediário entre clientes, barrels, downloaders e queue.

Responsável por:

1. Coordenação dos componentes
    - Registo e gestão dos barrels ativos
    - Registo e distribuição de tarefas para os barrels
2. Gestão das pesquisas
    - Recebe requisições de clientes
    - Envia as pesquisas para barrels disponíveis
    - Implementa a lógica de *failover* automático
3. Gestão da indexação
    - Gere a fila de URLs para indexação
    - Evita a duplicação de URLs já processados
    - Distribui URLs para Downloaders

Estruturas de Dados Principais:

Lista de Barrels registrados:

```
private final ArrayList<BarrelsINTER> barrels
```

URLs já processados:

```
private final Set<String> urlsIndexados;
```

Fila de URLs para indexação:

```
private QueueInterface urlQueue
```

Downloaders registrados:

```
private ArrayList<DownloaderINTER> downloaders
```

Dicionário barrel - nome do barrel:

```
private Map<BarrelsINTER, String> barrelNames
```

Métodos:

A classe inclui os métodos:

1. Registo de componentes
   - registerBarrel: Regista um novo Barrel e sincroniza com dados existentes, se necessário.
   - unregisterBarrel: Remove um Barrel do sistema e do RMI Registry.
   - registerDownloader: Regista um downloader para processamento de URLs.
   - registerQueue: Regista uma fila de URLs para a Gateway.
2. Pesquisas e consultas
   - searchWord: Realiza a pesquisa de uma palavra no índice invertido.
   - obterPaginasApontamPara: Obtém as páginas que apontam para um determinado URL.
3. Gestão da fila e indexação de páginas
   - getQueue: Retorna a fila de URLs do sistema.
   - putNew: Adiciona um novo URL à fila, se ainda não tiver sido processada.
   - addToIndex: Adiciona uma palavra e o URL correspondente ao índice invertido.
   - getNextURL: Retorna o próximo URL da fila para processamento.
4. Gestão dos barrels
   - getBarrel: Obtém um Barrel ativo para operações.
   - matarBarrel: Remove o registo de um barrel do sistema e do RMI Registry.
   - getAllBarrels: Retorna todos os barrels atualmente registados.

5. Inicialização do serviço
    - main: Método principal para iniciar a Gateway e registá-lo no RMI Registry. Configura o host e porta, cria o registro RMI e vincula a instância da gateway para estar disponível remotamente.

Interface:

A sua interface inclui os métodos:

- List<String[]> searchWord(String word) throws RemoteException;
- void putNew(String url) throws RemoteException;
- List<String> obterPaginasApontamPara(String url) throws RemoteException;
- BarrelsINTER getBarrel() throws RemoteException;

Comunicação com outros componentes:

1. Barrels
    - Balanceamento de carga entre réplicas;
    - Sincronização de novos barrels;
    - Monitorização da “vida” do barrel através da invocação do método do barrel: ping().
2. Downloaders
    - Distribuição de URLs para processamento;
    - Gestão da fila e prevenção de URL duplicados.

Tratamento de falhas:

1. Tolerância a falhas dos barrels
    - Deteção de barrels inativos;
    - Remoção do registry RMI;
    - Redirecionamento automático de pedidos para réplicas.
2. Consistência dos dados
    - Sincronização de novos barrels com dados existentes;
    - Verificação de URLs já processados.

Downloaders

Descrição geral:

O downloader é o componente responsável por processar páginas web. Ele atua como um *worker* que consome URLs da fila, extrai conteúdo e envia informações para indexação nos barrels.

Responsável por:

1. Processamento de páginas web
   - Download de conteúdo HTML usando a biblioteca Jsoup;
   - Extração estruturada de:
     - Títulos;
     - Citações (primeiro parágrafo ou *meta description*);
     - Links (hiperligações para outras páginas);
     - Texto completo para indexação.
2. Gestão de URLs
   - Transformação de URLs relativas em absolutas;
   - Adição de novos URLs descobertos à fila de processamento;
   - Garantia de que um URL é processado uma única vez.

3. Pré-processamento de Conteúdo
   - Deteção de língua;
   - Filtragem de *stop words*;
   - Normalização de texto (minúsculas, remoção de pontuação).

Estruturas de Dados Principais:

Identificador único do downloader (ex.: "Downloader1"):
    String downloader_name

Referência RMI para a gateway, usada para obter URLs e enviar resultados:
    GatewayINTER gateway

Fila de URLs a serem processados (gerida pela gateway):
    QueueInterface urlQueue

Dicionário para contar em quantas páginas uma palavra está presente:
    Map<String, AtomicInteger> localWordCount

Métodos principais

1. Inicialização e Registo
   - main(String[] args): Ponto de entrada do downloader. Regista-se no RMI Registry e inicia o processamento.

- criarDownloader(String nome): Cria uma instância do downloader e regista-a na gateway. Obtém a referência para a fila de URLs.

2. Processamento de URLs
   - executar(): Método principal em *loop* que:
     - Pede um URL da fila;
     - Verifica se o URL é válido (evita null ou URLs malformatados);
     - Chama processarPagina(url) para extrair conteúdo.
   - processarPagina(String url):
     - Passos:
       1. Verifica se o URL já foi indexado (barrel.containsURL(url)).
       2. Baixa a página, usando Jsoup.
       3. Extrai:
          - Título (doc.title());
          - Citação (primeiro <p> ou *meta description*);
          - Links (doc.select("a[href]"));
          - Texto (doc.text()).
       4. Normaliza o texto (minúsculas, remove pontuação);
       5. Envia para indexação (gateway.addToIndex());
       6. Conta a frequência de cada palavra
   - enviarEstatisticasParaBarrels(BarrelsINTER barrel): envia as estatísticas para um barrel disponível;
   - transformarUrlAbsoluta(String baseUrl, String href): Converte URLs relativas em absolutas e ignora URLs inválidos;
   - put_url(String url): Adiciona um novo URL à fila de indexação (gateway.putNew(url)).

Interface:

A sua interface inclui o método:

- void processarPagina(String url) throws RemoteException;

Comunicação com outros componentes:

1. Gateway
   - Obtém URLs da fila (getNextURL);
   - Envia palavras indexadas (addToIndex);
   - Adiciona novos URLs (putNew).
2. Barrels
   - Verifica se um URL já foi indexado (containsURL);

Tratamento de falhas:

1. URLs Inválidos
   - Ignora URLs mal formatados.
2. Falhas de ligação

- Trata erros de download (IOException) e continua a processar outros URLs.

3. Fila Vazia
   - Se não houver URLs, aguarda 1 segundo antes de tentar novamente.

Barrels

Descrição geral:

Os barrels são componentes essenciais no sistema deste motor de pesquisa, são os servidores de armazenamento distribuído para o índice invertido e dados relacionados.

Responsável por:

1. Armazenamento do Índice Invertido:
   - Um dos atributos é um HashMap para o índice invertido;
   - Cada chave corresponde a uma palavra cujo valor é a lista de páginas onde ela aparece;
   - Para cada página, o barrel armazena: URL, título, citação e links para outras páginas.

2. Gestão de links que apontam para outras páginas:
   - Tem como atributo um HashMap<String,HashSet<String>> para rastrear quais páginas apontam para cada URL. Isto é usado para calcular a relevância das páginas nos resultados das pesquisas.

3. Deteção de *Stop Words:*
   - Implementa um sistema para identificar *stop words*;
   - Recebe a frequência local de palavras por página do downloader e calcula as palavras mais frequentes;
   - Remove as palavras mais comuns (top 5%) do índice.
     - Num motor com mais recursos, teríamos escolhido uma percentagem maior mas, como estamos a tratar, relativamente, poucas páginas, considerámos que uma percentagem maior iria retirar demasiadas palavras do índice.

Estruturas de Dados

1. Índice principal que mapeia palavras para listas de páginas (URL, título, citação, links):

   private HashMap<String, ArrayList<String[]>> indiceInvertido

2. Referência RMI para a gateway, usada para obter URLs e enviar resultados:

   GatewayINTER gateway

3. Nome com que o barrel se vai registar:

   String name

4. Registo de quantas páginas apontam para cada URL (usado para calcular relevância):
   private HashMap<String, HashSet<String>> ponteiros

5. *Lock* para sincronizar acesso ao ficheiro que guarda o índice enquanto a informação é enviada para ele:
   private static final Object fileLock

6. Caminho do ficheiro para onde as *stop words* são guardadas:
   private final String ficheiroStopWords

7. Lista de stop words:
   Set<String> stopWords

8. Percentagem das palavras do índice a serem removidas (stop words):
   double STOP_WORD_PERCENTAGE

9. Caminho do ficheiro para onde o índice é guardado:
   String ficheiroURLbarrels

10. Dicionário com informação sobre a frequência das palavras:
    Map <String, AtomicInteger> contagens

Métodos:

A classe inclui os métodos:

1. Construtores e Inicialização:

- Barrels(String name): Construtor que inicializa o barrel com um nome específico, carrega o índice invertido e stopWords, e configura um *shutdown hook* para salvar dados antes de terminar.
- criarbarrel(String barrel_nome, int gateway_port): Método que cria um novo barrel e regista-o na gateway via RMI.

2. Gestão de Índice Invertido:

- addToIndex(String word, String url, String titulo, String citacao, List<String> links): Adiciona um URL associado a uma palavra ao índice invertido e propaga para outros barrels.
- mandarIndex(String word, String url, String titulo, String citacao, List<String> links): Recebe dados de indexação de outros barrels.
- carregarIndice(): Carrega o índice invertido e stopWords de ficheiros locais.
- reconstruirPonteiros(): Reconstrói o mapa de ponteiros a partir do índice carregado.
- salvar(): Guarda o índice e *stop words* nos respectivos ficheiros.

3. Pesquisa e Consultas:

- top10(String termos): Retorna as 10 páginas mais relevantes para os termos da pesquisa.
- obterpaginaurlponteiros(String url): Obtém páginas que apontam para um URL específico.
- obterrelevancia(String url): Calcula a relevância de uma página baseada noutros links recebidos.
- contarTermosNaPagina(String[] pagina, String[] termos): Auxiliar para contar ocorrências de termos em uma página.

4. Gestão de *stop words*:

- receberContagemPalavras(Map<String, AtomicInteger> novasContagens): Recebe contagens de frequências de palavras para cálculo de stop words.
- recalculateStopWords(): Recalcula as stop words baseado nas contagens.
- removeStopWordsFromIndex(): Remove stop words do índice invertido.
- syncStopWordsWithOtherBarrels(Set<String> newStopWords): Sincroniza stop words com outros barrels.
- updateStopWords(Set<String> newStopWords): Atualiza a lista do barrel de stop words.

5. Sincronização e Comunicação

- carregarDados(Map<String, ArrayList<String[]>> indice, Set<String> stopWords): Carrega dados recebidos de outro barrel quando o barrel é iniciado.
- getIndiceInvertido(): Retorna o índice invertido atual.
- getStopWords(): Retorna as stop words atuais.
- ping(): Método de verificação de atividade.

A sua interface inclui os métodos:

- void addToIndex(String word, String url, String titulo, String citacao, List<String> linha) throws RemoteException;
- boolean containsURL(String url) throws RemoteExceptions;
- List<String> top10(String termos) throws RemoteException;
- List<String> obterpaginaurlponteiros(String url) throws RemoteException;
- void ping() throws RemoteException;
- String getName() throws RemoteException;
- void mandarIndex(String word, String url, String titulo, String citacao, List<String> linha) throws RemoteException;
- Set<String> getStopWords() throws RemoteException;
- void carregarDados (Map <String, ArrayList <String []>> indice, Set <String>stopWords) throws RemoteException;
- Map<String, ArrayList<String>> getIndiceInvertido() throws RemoteException;
- void updateStopWords(Set<String> newStopWords) throws RemoteException;
- void receberContagemPalavras(Map<String, AtomicInteger> contagens) throws RemoteException.

Comunicação com outros componentes

1. Gateway
    - Os barrels registram-se na Gateway através de RMI.
2. Downloaders
    - Os barrels recebem atualizações dos downloaders;
    - Propagam essas atualizações para outros Barrels.
3. Outros Barrels
    - Mantêm consistência através de reliable multicast;
    - Quando um novo barrel entra no sistema, ele recebe os dados atualizados de um outro barrel.

Tratamento de falhas

1. Barrels inativos
    - Caso um barrel falhe, vai ser lançada uma exceção quando a gateway chama o método ping(), presente na interface do barrel, e o barrel inativo é removido da lista de barrels da gateway e do registo RMI.
2. Persistência dos dados em caso de falhas
    - O barrel carrega os dados do índice e das *stop words* a partir dos seus ficheiros locais ao iniciar. Como as réplicas mantêm estruturas de dados idênticas (atualizadas através de *reliable multicast*), o sistema permanece funcional mesmo em caso de falha de um barrel.
3. Concorrência
    - Estruturas como o índice invertido, que é acessado por múltiplas operações, sendo modificado e iterado durante as mesmas, está protegido através do *synchronized*.

<u>Queue</u>

Descrição geral:

A URLqueue é uma fila partilhada de URLs para a indexação de páginas web. Implementada como um serviço RMI, garante a gestão organizada de URLs a serem processados pelos downloaders, garantindo escalabilidade.

Responsável por:

1. Armazenamento de URLs para indexação.
2. Sincronização entre a gateway (que envia os URLs aos downloaders) e os downloaders (consumidores):
    - Garante a integridade dos dados: evita duplicação ou perda de URLs durante acesso;
    - URLs persistem na fila mesmo se um downloader falhar.
3. Prevenção de sobrecarga do sistema (limite máximo configurável de URLs na fila).

Estruturas de dados principais

1. Fila para armazenar URLs: LinkedBlockingQueue<String> urls
2. Tamanho máximo da fila: int max_size
3. Referência para a Gateway (registo RMI): GatewayINTER gateway

Métodos:

A classe inclui os métodos:

Gestão da fila:

- putURL: Adiciona um URL à fila. Se a queue não estiver cheia, o URL será adicionado à fila. Após adicionar o URL, a queue é salva.
- getURL: Obtém e remove o próximo URL da fila. Este método bloqueia a execução até que um URL esteja disponível na fila. Se a fila estiver vazia, o método aguarda até que um novo URL seja adicionado. Após remover o URL da fila, ela é marcada como processada e a fila é guardada no ficheiro.
- getQueueSize: Retorna o número de URLs presentes na fila.
- getMaxSize: Retorna o tamanho máximo permitido para a queue.

Inicialização:

- createQueue:Cria e regista uma nova instância da fila de URLs no registo RMI. Este metodo cria uma fila com o tamanho máximo de 1000 URLs e regista a fila no registo RMI para que outros componentes possam acessá-la remotamente.
- main: Método principal que cria e regista a fila no registro RMI

Interface

A sua interface inclui os métodos:

- void putURL(String url) throws RemoteException;
- String getURL() throws RemoteException, InterruptedException;
- int getQueueSize() throws RemoteException, InterruptedException;
- int getMaxSize() throws RemoteException, InterruptedException;

Comunicação com outros componentes

1. Gateway
    - Adiciona URLs através do método putURL .
2. Downloaders

- Consomem URLs através do método getURL para processamento.

Tratamento de falhas

1. Fila cheia
    - putURL ignora novos URLs além do max_size
2. Fila vazia
    - O método getURL bloqueia downloaders até que novos URLs estejam disponíveis
3. Concorrência
    - synchronized e LinkedBlockingQueue evitam a corrupção de dados.

Cliente

Descrição geral:

O cliente é a interface do utilizador do sistema, permitindo interagir com todas as funcionalidades do motor de pesquisa através de um menu interativo. Implementado como um cliente RMI, comunica-se exclusivamente com a Gateway para realizar operações.

Responsável por:

1. Interação com o utilizador:
    - Apresenta um menu com as funcionalidades do sistema;
    - Processa inputs do utilizador (URLs, termos de pesquisa, etc.).

2. Comunicação com a gateway:
    - Envia pedidos de indexação (putNew);
    - Solicita pesquisas (searchWord);
    - Consulta ligações entre páginas (obterPaginasApontamPara).

3. Apresentação de resultados:
    - Mostra resultados de pesquisa paginados (10 em 10);
    - Exibe informações detalhadas (título, URL, citação).

Estruturas de Dados Principais:

1. Referência remota para a Gateway (obtida através do RMI Registry):
   GatewayINTER gateway
2. Leitor de inputs do utilizador:
   Scanner sc

Métodos Principais:

1. Gestão do menú
    - exibirMenu(): Mostra as opções disponíveis e redireciona para as funcionalidades correspondentes.

      Opções:
        - Indexar URLs;
        - Pesquisar páginas por importância;
        - Consultar ligações para uma página;
        - Visualizar palavras vazias (*stop words*);
        - Sair.
2. Funcionalidades
    - solicitarURL(): Permite ao utilizador inserir um URL para indexação (através da inovação do método gateway.putNew).
    - pesquisarconjtermos(): Realiza pesquisas com múltiplos termos e mostra resultados paginados.
    - consultarligacoespagina(): Lista páginas que linkam para um URL específico.

Interface

A sua interface inclui os métodos:

1. void solicitarURL() throws RemoteException;
2. void pesquisarconjtermos() throws RemoteException;
3. void consultarligacoespagina() throws RemoteException;

Comunicação com Outros Componentes

1. Gateway

- Todas as operações são redirecionadas para a Gateway através do seu registo RMI.

Tratamento de falhas

1. Erros de ligação:
    - É lançada uma RemoteException caso a gateway não esteja disponível.
2. Inputs inválidos:
    - Valida entradas do utilizador.

Passando agora para a parte do frontend:

BackendRMICliente

Descrição geral:

Esta classe atua como intermediária, sendo o ponto de comunicação entre a aplicação e o backend através de RMI. Esta arquitetura facilita a interação com o backend de forma escalável, encapsulando toda a lógica de comunicação remota.

Funcionalidades:

Esta classe oferece as seguintes funcionalidades principais:

- Pesquisar termos: Permite que o cliente faça pesquisas utilizando os termos que este deseja, retornando uma lista de resultados.
- Indexar URLs: Permite que um URL seja indexado no sistema.
- Consulta de links com ligação para outras páginas: Permite acessar todos os links que apontam para uma página específica.
- Obtenção de stopwords: Permite que o cliente recupere as palavras mais comuns.

Métodos:

Todos estes métodos invocam métodos do backend. A classe inclui os métodos:

1. Indexar URL

   `indexarURL`- permite ao cliente indexar um novo URL no sistema.

2. Pesquisas

   `search`- permite ao cliente realizar uma pesquisa no sistema.

   `consultarLinks`- permite consultar todos os links que apontam para um URL fornecido pelo cliente.

3. Stop words

   `getStopWords`- Retorna as stop words presentes no sistema.

OpenRouterAPI

Descrição geral:

Esta classe interage com a API OpenRouter, que utiliza o modelo microsoft/phi-4-reasoning-plus:free para gerar análises contextuais com base no termo pesquisado e nos resultados de pesquisa fornecidos fornecidos pelo backend. Ela é essencial para a integração com a inteligência artificial externa.

Funcionalidades:

Esta classe oferece a seguinte funcionalidade principal: gerar análises contextuais.

Métodos:

Com apenas um método, `gerarAnaliseContextual`, é responsável por gerar a análise contextualizada utilizando a API OpenRouter. O método recebe uma string (termo de pesquisa) e a lista de resultados obtidos dessa pesquisa, organiza esses dados num formato adequado para a API e então, a API processa e retorna uma análise.

Views

Descrição geral:

Representam a camada de aplicação de apresentação, ou seja, são as páginas web com as quais o cliente interage. São responsáveis por exibir os dados e permitir que o cliente interaja com o sistema de forma intuitiva.

Funcionalidades:

As views oferecem as seguintes funcionalidades ao cliente:

1. Página inicial:

   Exibe uma caixa de pesquisa para o cliente inserir o termo de pesquisa, botões para aceder a outras funcionalidades, como consultar ligações para uma página, consultar stop words e indexar urls.

2. Página de indexação de URL:

   Permite que o cliente insira um URL para ser indexado ao sistema.

3. Página com hacker news
4. Página de análise com IA:

Exibe o resultado da análise feita pela IA.

5. Página de Stop Words

Views:

As views interagem com o controlador através de métodos que processam as requisições do cliente.

1. `index`- Página inicial, que serve como ponto de entrada para o cliente.
2. `pesquisarURL`- Permite ao usuário inserir um URL para encontrar uma lista de páginas com ligação para uma página específica
3. `search_results`- Esta página exibe os resultados da pesquisa baseados no termo inserido pelo cliente na página inicial
4. `results_url`- Exibe todos os links encontrados que apontam para um URL específico
5. `checkStopwords`- Exibe uma lista de stop words
6. `indexarURL`- Página onde o cliente pode inserir um URL para ser indexado no sistema.
7. `TopStoriesResultado`- Exibe a quantidade de links que foram indexados
8. `chat_completions`- Página que exibe a análise gerada pela IA

Métodos Post e get:

O método GET é usado principalmente para recuperar informações do servidor.

O método POST é utilizado para enviar dados do cliente para o servidor.

Controllers

Descrição geral:

Os controladores implementam a lógica responsável por gerir a interação entre o frontend e o backend.

Funcionalidades:

Oferecem as seguintes funcionalidades:

1. Pesquisa de URLs
2. Indexação de URLs
3. Visualização de Stop words
4. Integração com Hacker news
5. Integração com OpenAI

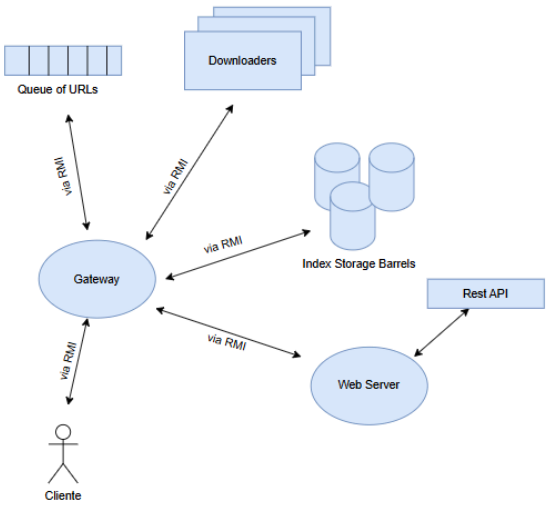
Métodos:

Os métodos são:

- GreetingController()
  Construtor que inicializa o BackendRMIcliente e estabelece a ligação com o backend remoto via RMI
- index(Model model)
  Método que exibe a página inicial
- linkForm()
  Exibe a página de pesquisa para indexar o URL
- linkResult(String url, Model model)
  Exibe os links que apontam para um URL específico
- search(String query, int page, Model model)
  Realiza a pesquisa de uma palavra e exibe os resultados paginados
- verStopWords(Model model)
  Exibe as stop words do sistema.
- indexarURLForm(Model model)
  Exibe a página para o cliente indexar o URL
- indexarURL (String url, Model model)
  Realiza a indexação de um URL no sistema
- hackerNewsTopStories(String search)
  Recupera as top stories do Hacker News e filtra com base nas palavras que o cliente inseriu na sua pesquisa
- indexarTopStories (String search, String confirm, Model model)
  Solicita a indexação das top stories do Hacker News com base no parâmetro de pesquisa fornecido e vai para a página onde será apresentado o resultado
- gerarAnaliseComIA (String search, Model model)
  Realiza a pesquisa e gera a análise contextual e o resultado será apresentado

Detalhes dos Componente

A imagem ilustra a arquitetura do sistema, como é a comunicação do fluxo de dados entre os principais componentes, já mencionados e falados em cima.



A estrutura das pastas, mostrada nas seguintes imagens, é organizada da seguinte maneira dentro da pasta BeeCurious_SD:

1. src\main\java: Contém o código da aplicação. Aqui é onde se encontra:
- o backend (search) ;
- a parte lógica do frontend, excluindo os ficheiros HTML(meta2).
2. src\main\resources: Responsável pelos recursos utilizados. Aqui encontram-se:
- as páginas HTML;
- o logotipo.

```
C:.
├───.idea
│   └───libraries
├───.mvn
│   └───wrapper
├───lib
├───out
│   └───production
│       └───BeeCurious_SD
│           └───main
│               └───resources
│                   ├───static
│                   └───templates
```

```
├───src
│   ├───main
│   │   ├───java
│   │   │   ├───meta2
│   │   │   └───search
│   │   └───resources
│   │       ├───static
│   │       │   └───images
│   │       └───templates
│   └───test
│       └───java
│           └───com
│               └───example
│                   └───demo
```

```
└───target
    ├───classes
    │   ├───main
    │   │   └───images
    │   └───templates
    ├───generated-sources
    │   └───annotations
    ├───generated-test-sources
    │   └───test-annotations
    ├───maven-status
    │   └───maven-compiler-plugin
    │       ├───compile
    │       │   └───default-compile
    │       └───testCompile
    │           └───default-testCompile
    └───test-classes
        └───com
            └───example
                └───demo
```

Integração com Frameworks

A implementação deste sistema baseia-se numa arquitetura que exige a integração de diferentes frameworks. Um dos pilares deste trabalho foi o uso de RPC/RMI, permitindo que métodos de objetos fossem chamados remotamente noutro local, possibilitando a comunicação entre a gateway, os downloaders e os storage barrels. Cada uma destas arquiteturas disponibiliza métodos acessíveis através das suas interfaces.

Na segunda meta foi integrada uma camada web utilizando o framework Spring Boot. Esta camada expõe funcionalidades através de REST APIs, permitindo a interação entre o cliente e o sistema (backend). O projeto faz uso de bibliotecas como Unirest para facilitar as HTTP a APIs externas (como o Hacker News) e o Gson, que foi essencial para enviar e receber dados em formato JSON, facilitando a integração com a OpenRouter AI. Além disso, são utilizadas ferramentas como o Jsoup para processar os dados e extrair informações relevantes de páginas web.

Ao receber uma requisição HTTP, o controlador no servidor Spring Boot atua como uma ponte entre a interface web e o sistema da primeira meta (backend),

comunicando com este via RMI. Desta forma, todas as funcionalidades anteriormente disponíveis, através da classe Cliente, foram expostas ao utilizador via web, mantendo a lógica do backend.

Embora existam opções como WebSockets para estatísticas em tempo real, esta abordagem não foi necessária uma vez que não necessitámos de exibir estatísticas/informações do sistema em tempo real.

Por fim, utilizamos também o Maven como ferramenta de gestão de dependências e de construção do projeto. Através de alguns comandos foi possível compilar, testar e executar a aplicação frontend, o que contribuiu para um fluxo mais eficiente.

Distribuição de Tarefas na Equipa

Durante a primeira meta, a Beatriz focou-se mais nas funcionalidades que envolvem o processamento de páginas por parte dos downloaders, como a indexação e a pesquisa de termos, enquanto a Diana se focou mais na parte da comunicação entre os componentes, como tratar as falhas de barrels e na deteção de *stop words*. Na segunda meta, a Beatriz focou-se na ligação do *backend* com o *frontend*, enquanto a Diana se focou mais em resolver bugs e na estética do motor de pesquisa.

Teste de Software

O objetivo deste teste é validar o comportamento de diversos componentes e funcionalidades do sistema.

Teste	**Resultado (pass/fail)**	**Output/Comentário**
Testar se o cliente aceita uma opção válida e ignora uma inválida.	Pass	Se o cliente inserir uma opção que não esteja entre as opções disponíveis, é exibida a seguinte mensagem de erro: Opção inválida.
Resultados ordenados por número de ligações recebidas de outras páginas	Pass	As páginas são ordenadas através de dois critérios: o primeiro é através da relevância (através de ponteiros) e, em caso de empate, pelo número de termos de pesquisa encontrados no conteúdo.
Consultar lista de páginas com ligações para uma página específica	Pass	Se não tiver URLs a apontar para este aparece a seguinte mensagem:Nenhuma página encontrada para este url Caso contrário aparece apenas os URL que apontam para essa página, sem nenhuma mensagem no terminal.

Aprendizagem computacional distribuída de *stop words* (1º teste)	Fail	Existia um erro de concorrência porque os diferentes processos estavam a tentar acessar o índice tanto para iterar sobre ele como para o modificar.
Aprendizagem computacional distribuída de *stop words* (2º teste)	Pass	Utilizámos o *synchronized* para resolver o problema anterior e agora os downloaders contam em quantas páginas cada palavra está presente e enviam as suas estatísticas para os barrels, que processam essas contagens locais e removem do índice as palavras com mais frequente do índice.
A informação é idêntica em todos os storage barrels (*reliable multicast*)	Pass	Cada vez que um barrel recebe informação, ele envia essa informação para os restantes barrels e quando um novo barrel é criado, além de carregar informação dos seus ficheiros locais, recebe nova informação de um barrel ativo.
Indexar URLs de fonte externa (p.ex., HackerNews) contendo os termos da pesquisa	Pass	No terminal vão aparecer vários logs de informação e, depois, uma mensagem acompanhada: se o termo de pesquisa aparece ou não. Além disso, no motor de pesquisa aparece uma mensagem de indexação.
Gerar um texto contextualizado com IA (OpenRouter)	Pass	Após carregarmos as dependências, o *output* é um texto gerado através dos termos de pesquisa e dos resultados que obtidos pelo motor de pesquisa.
O projeto corre distribuído por várias máquinas	Pass	Alterando o IP para aquele onde queremos que o registo seja criado, o projeto corre gateway, barrels, downloaders e cliente de forma distribuída.
Verificar se os downloaders respondem corretamente quando a gateway não está conectada	Pass	Se a gateway não estiver a correr, aparecerá um erro a informar que a conexão foi recusada, e a seguinte mensagem será exibida: - - downloader (nome que deu quando registou o downloader) parou
Verificar se um novo storage barrel se sincroniza corretamente com os restantes	Pass	Quando acrescentamos um barrel, o método o sincronizar com outro barrel ativo é chamado e aparece a seguinte mensagem, caso não haja nenhuma falha fatal ao sistema: Dados sincronizados com sucesso.

Os downloaders executam em paralelo (cada um sobre o seu URL)	Pass	Cada downloader é um processo diferente e cada downloader mostra quais links retirou da fila. Eles retiram da fila links diferentes, processam diferentes páginas, pelo que a carga é distribuída.
Balanceamento da carga nas pesquisas sobre os storage barrels	Pass	Na gateway, aparece a mensagem “Barrel ativo + nome” sempre que a gateway escolhe um barrel para realizar um pedido e o próximo barrel escolhido é sempre o que corresponder ao próximo índice da lista.
Indexar novo URL introduzido por utilizador	Pass	O barrel mostra qual URL foi adicionado e a que palavra foi associado.
Indexar iterativamente ou recursivamente todos os URLs encontrados.	Pass	O downloader mostra quais os links extraídos de uma determinada página e a queue mostra que esses links foram adicionados à fila para serem processados.
Serviço correto se funcionar pelo menos um storage barrel e a gateway (1ºteste)	Fail	Erros do tipo IndexOutOfBounds ocorriam porque o índice era atualizado antes da seleção do barrel. Correção: atualizar o índice só validar o barrel.
Serviço correto se funcionar pelo menos um storage barrel e a gateway (2ºteste)	Pass	Índice atualizado pós-validação do barrel ativo. Gateway loga: barrels inativos, remoções e próximo índice.
Pesquisar páginas que contenham um conjunto de palavras	Pass	Um barrel ativo percorre o seu índice para encontrar os termos da pesquisa e devolve os valores associados a essas palavras.

Guia de Execução do Sistema

Pré-requisitos

- IDE ou editor de texto (simplificar a execução do código);
- Java 21.0.5 +

- Maven 3.9.6 +

Passo a passo

Para correr o . Aqui estão os passos que deve seguir:

1. Inicialização dos componentes: em todos tem de estar no diretório src/
2. Corra a gateway, barrels, downloaders e queue.
    - Barrel

      Pode iniciar múltiplos barrels, mas tem de variar o número do final do nome de cada um destes (onde está o #).

      ```
      java main.java.search.Barrels barrel# 8183
      ```

    - Downloader

      Para os downloaders faça isto:

      ```
      java -cp ".\main\jsoup-1.19.1.jar;."
      main.java.search.Downloader downloader#
      ```

      Pode adicionar vários downloaders, mudando o número no final do nome de cada um.
3. Dentro do diretório BeeCurious_SD, execute o seguinte comando:

   ```
   mvn spring-boot:run
   ```

4. Acessar pelo navegador

   Com os passos anteriores feitos, acesse ao http://IP:8080/ no seu navegador.

Conclusão

Este trabalho demonstra um motor de pesquisa de páginas web com funcionalidades completas que cumprem o objetivo deste trabalho. Com base neste documento, qualquer pessoa consegue perceber a estrutura do trabalho, desde a arquitetura geral do sistema até aos detalhes da implementação de cada componente.

Além disso, todo o nosso código está comentado com Javadoc, o que permite a qualquer pessoa compreender facilmente a estrutura, o funcionamento das funções e métodos e a interação entre os diferentes componentes.